

COMP3026E Distributed Systems

11. Scalable Causal Consistency

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen

Consistency Hierarchy (Review)



Linearizability

e.g., RAFT



Sequential Consistency



Causal+ Consistency

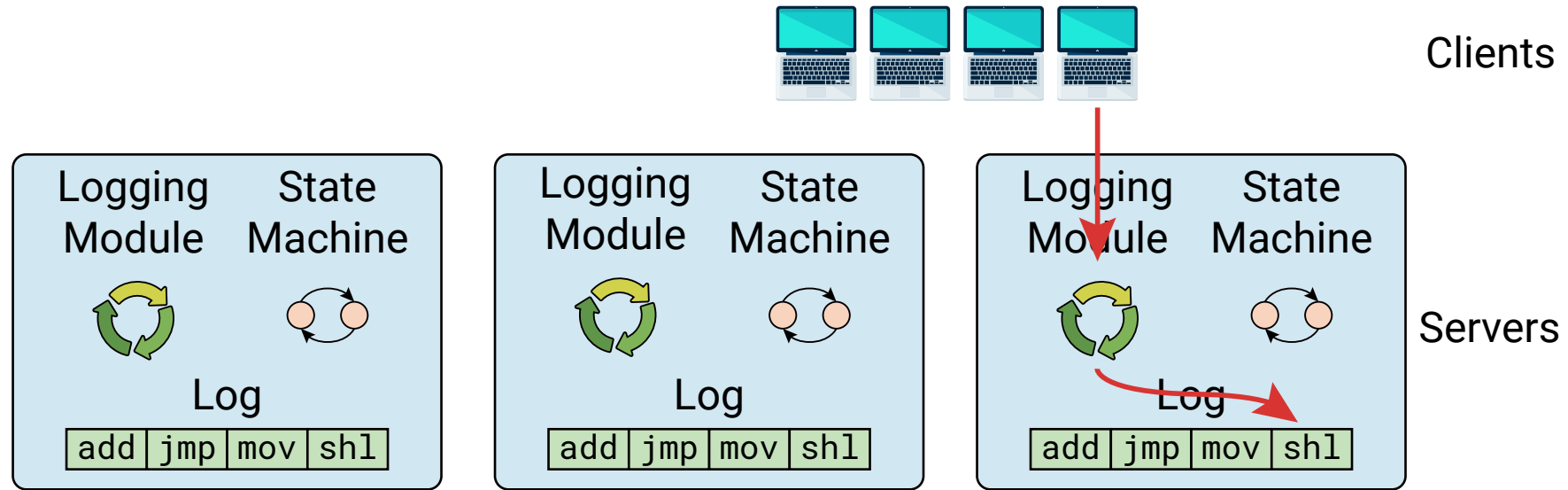
e.g., Bayou



Eventual Consistency

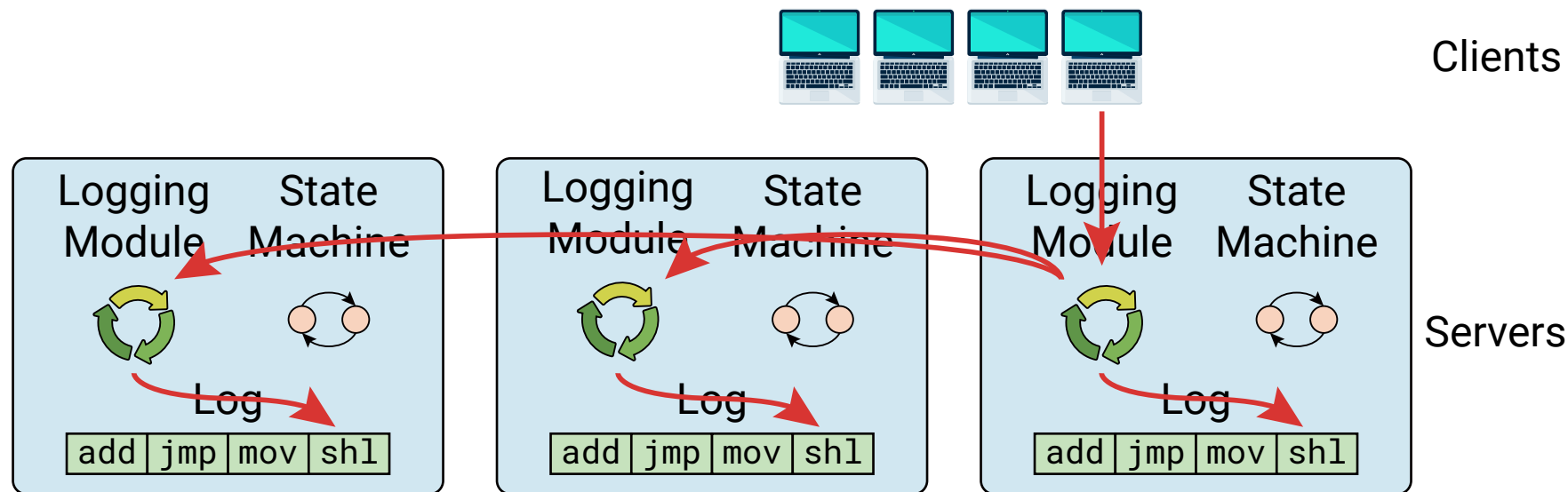
e.g., Dynamo

Causal Consistency Within Replicated Systems



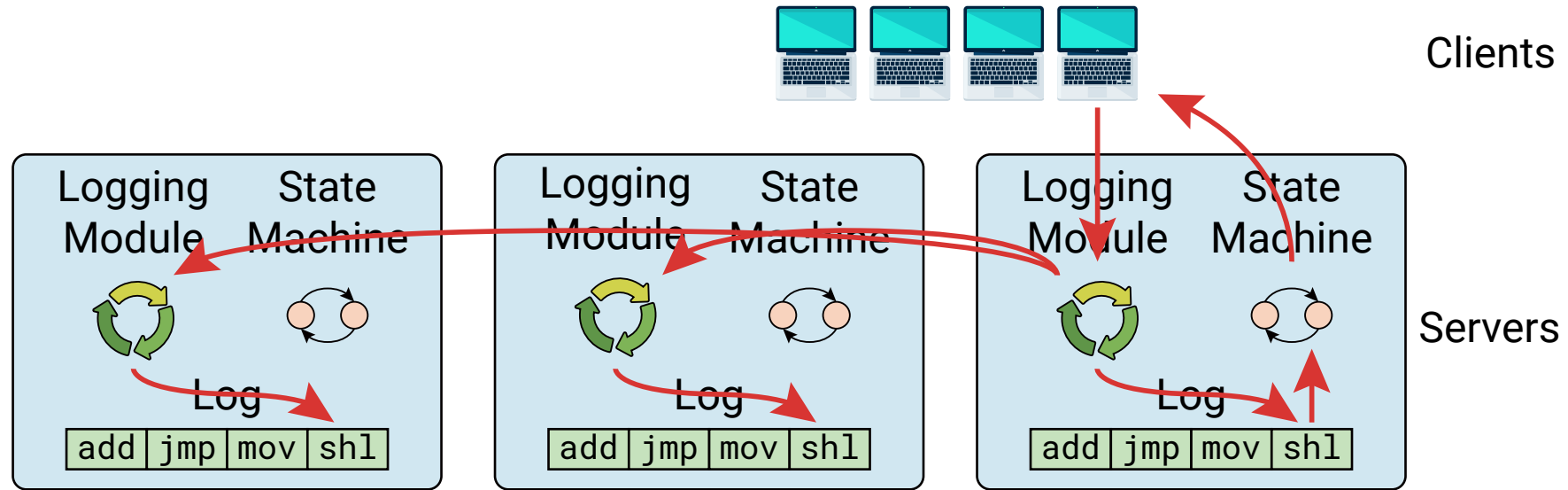
- **Linearizability** / sequential: Eager replication
- Trades off low-latency for consistency

Causal Consistency Within Replicated Systems



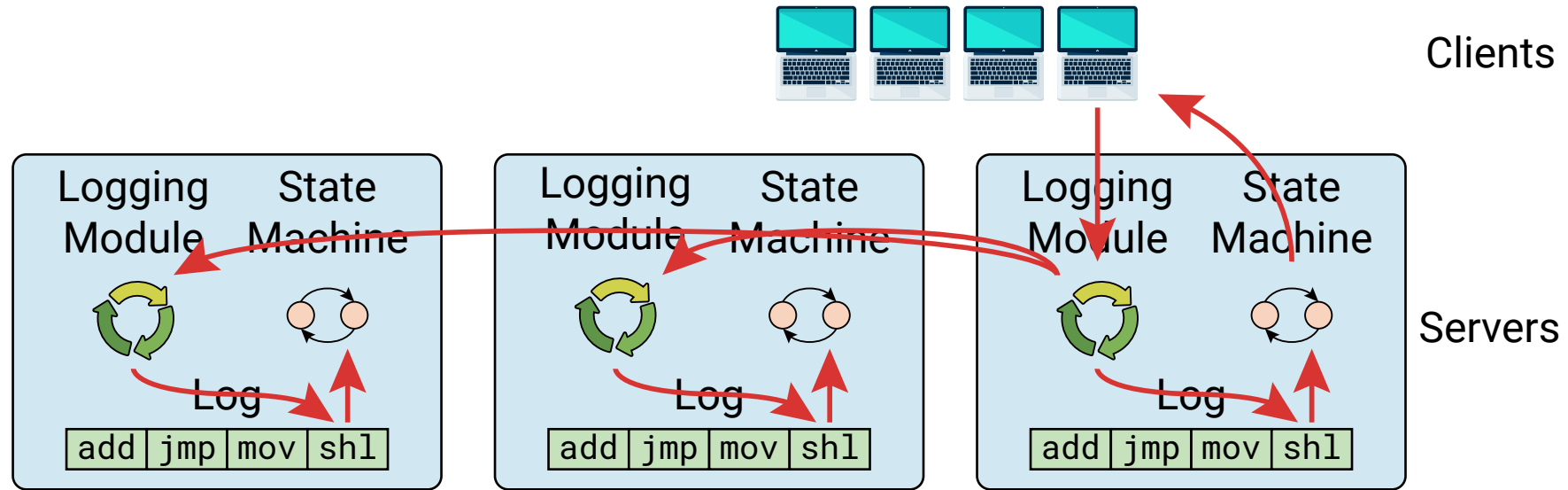
- **Linearizability** / sequential: Eager replication
- Trades off low-latency for consistency

Causal Consistency Within Replicated Systems



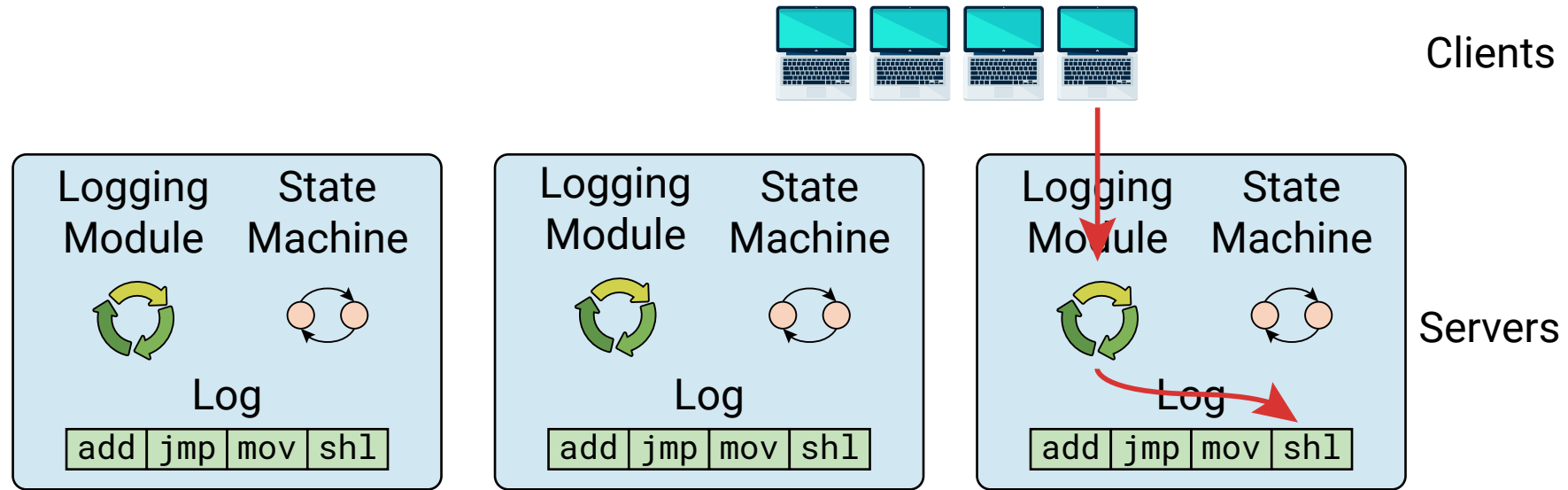
- **Linearizability** / sequential: Eager replication
- Trades off low-latency for consistency

Causal Consistency Within Replicated Systems



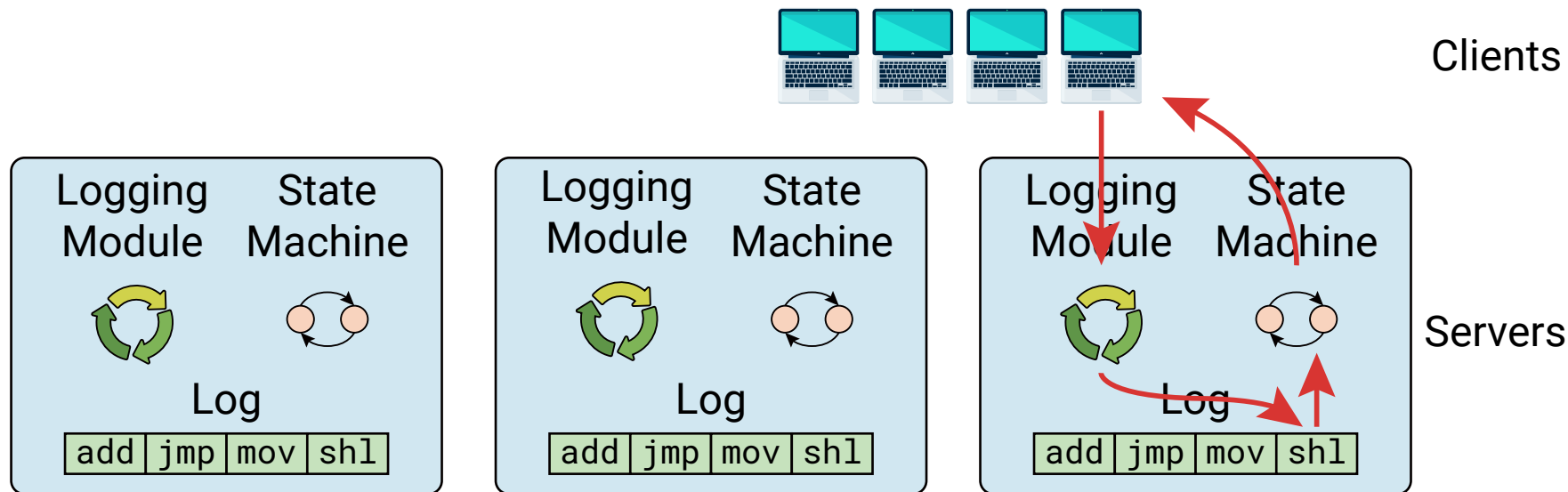
- **Linearizability** / sequential: Eager replication
- Trades off low-latency for consistency

Causal Consistency Within Replicated Systems



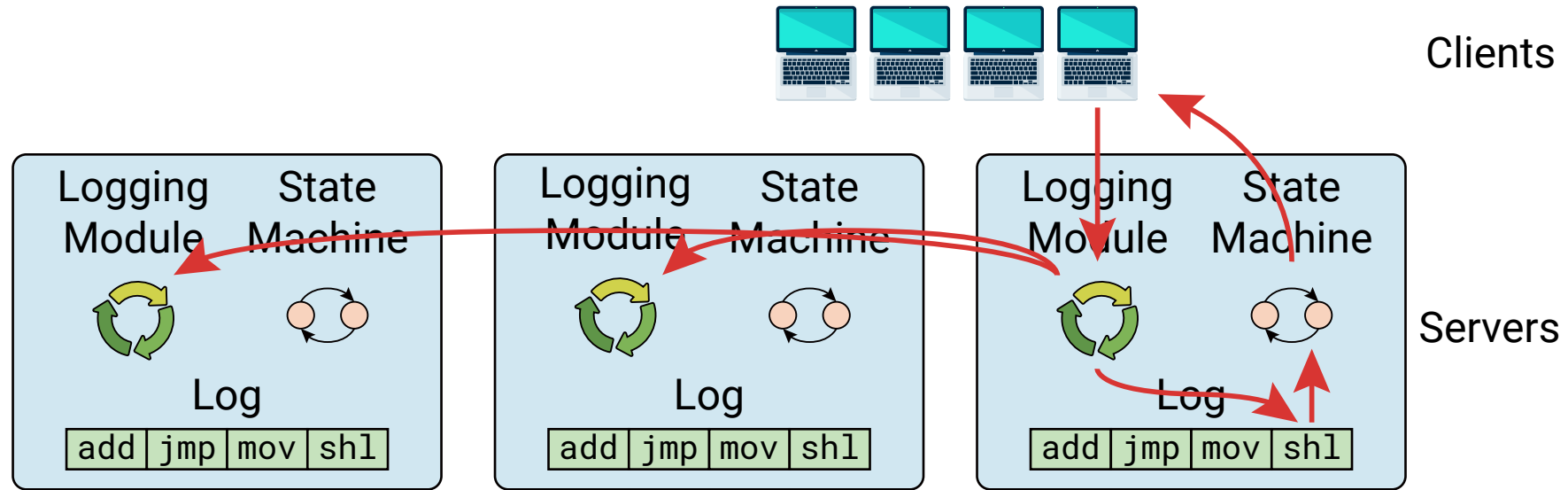
- **Causal consistency**: lazy replication
- Trades off consistency for low-latency
- Operations may be lost if failure before replication

Causal Consistency Within Replicated Systems



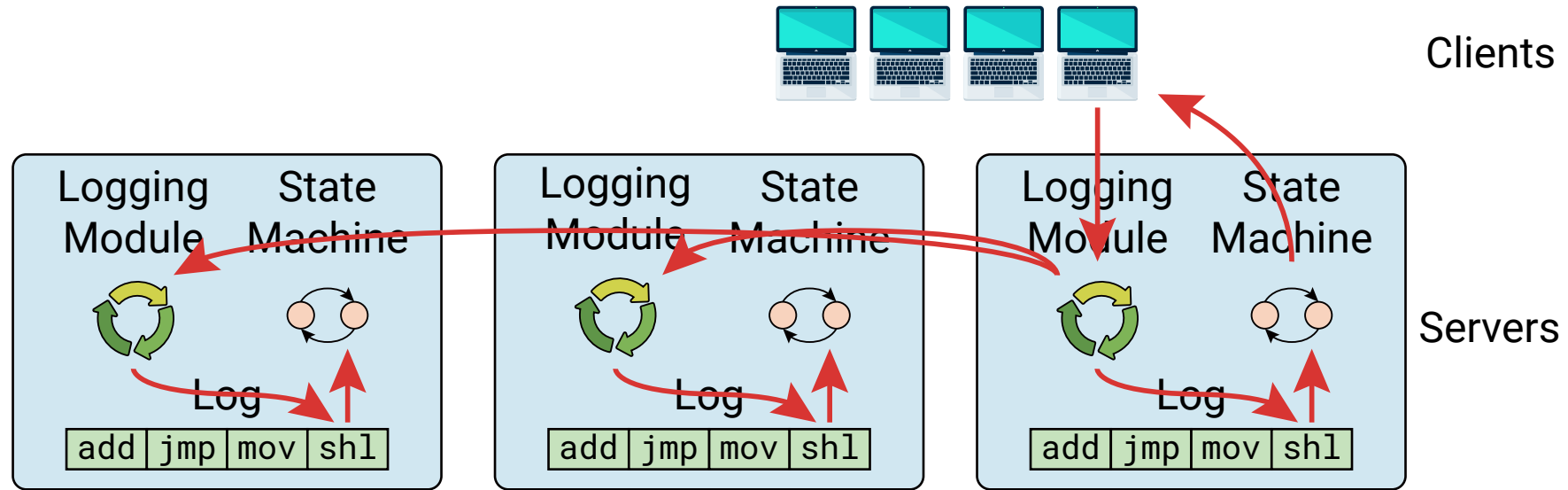
- **Causal consistency**: lazy replication
- Trades off consistency for low-latency
- Operations may be lost if failure before replication

Causal Consistency Within Replicated Systems



- **Causal consistency**: lazy replication
- Trades off consistency for low-latency
- Operations may be lost if failure before replication

Causal Consistency Within Replicated Systems



- **Causal consistency**: lazy replication
- Trades off consistency for low-latency
- Operations may be lost if failure before replication

Consistency vs Scalability



- **Scalability**: Adding more machines allows more data to be stored and more operations to be handled!

System	Consistency	Scalable?
Paxos / RAFT	Linearizable	
Bayou	Causal+	

Consistency vs Scalability



- **Scalability**: Adding more machines allows more data to be stored and more operations to be handled!

System	Consistency	Scalable?
Paxos / RAFT	Linearizable	No
Bayou	Causal+	

Consistency vs Scalability



- **Scalability**: Adding more machines allows more data to be stored and more operations to be handled!

System	Consistency	Scalable?
Paxos / RAFT	Linearizable	No
Bayou	Causal+	No

Consistency vs Scalability



- **Scalability**: Adding more machines allows more data to be stored and more operations to be handled!

System	Consistency	Scalable?
Paxos / RAFT	Linearizable	No
Bayou	Causal+	No

It's time to think about scalability!

Consistency vs Scalability



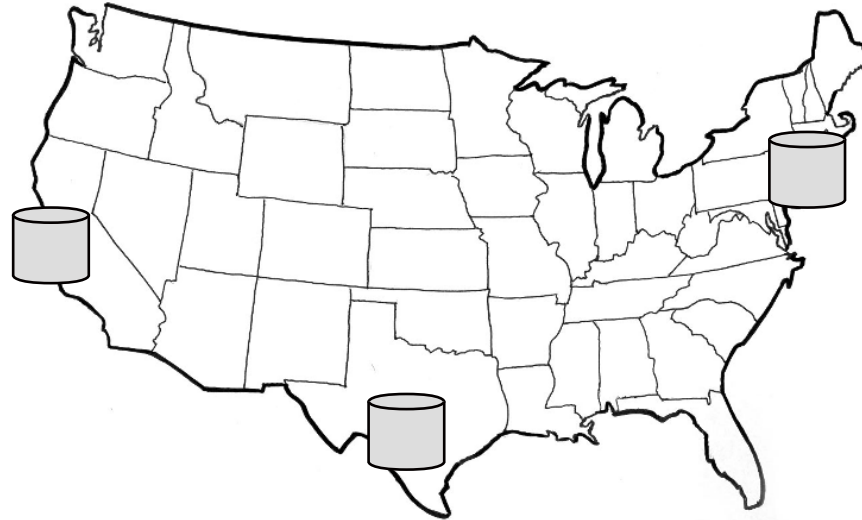
- **Scalability**: Adding more machines allows more data to be stored and more operations to be handled!

System	Consistency	Scalable?
Paxos / RAFT	Linearizable	No
Bayou	Causal+	No
COPS	Causal+	Yes

COPS: Cluster of Order-Preserving Systems

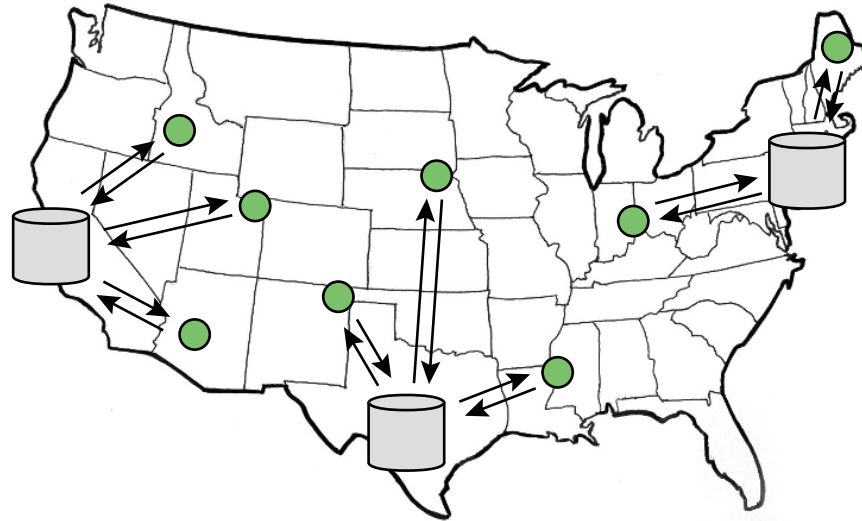
Scalable Causal Consistency for Geo-Replicated Storage

Geo-Replicated Storage

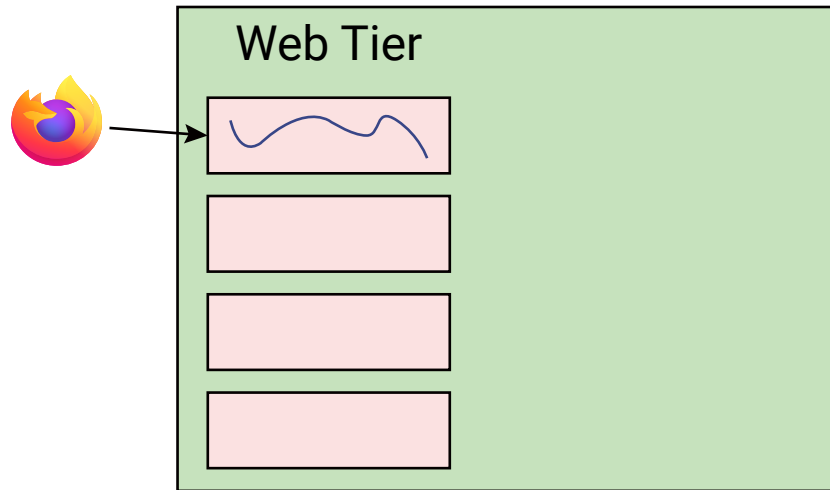


Geo-Replicated Storage

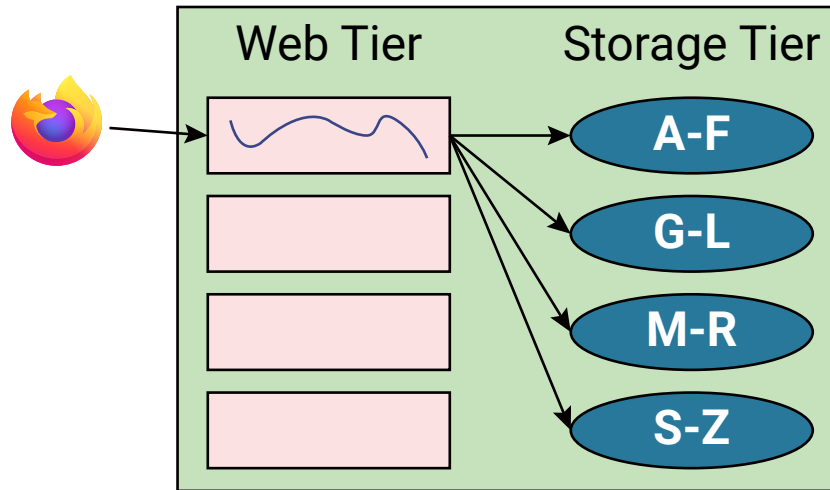
- serves requests quickly



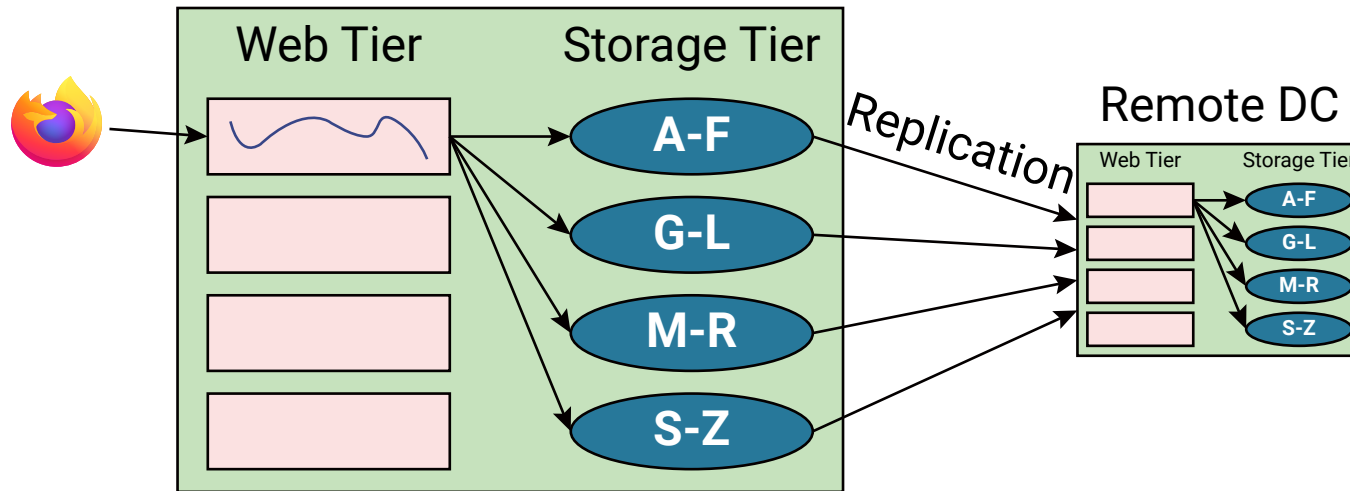
Inside the Datacenter



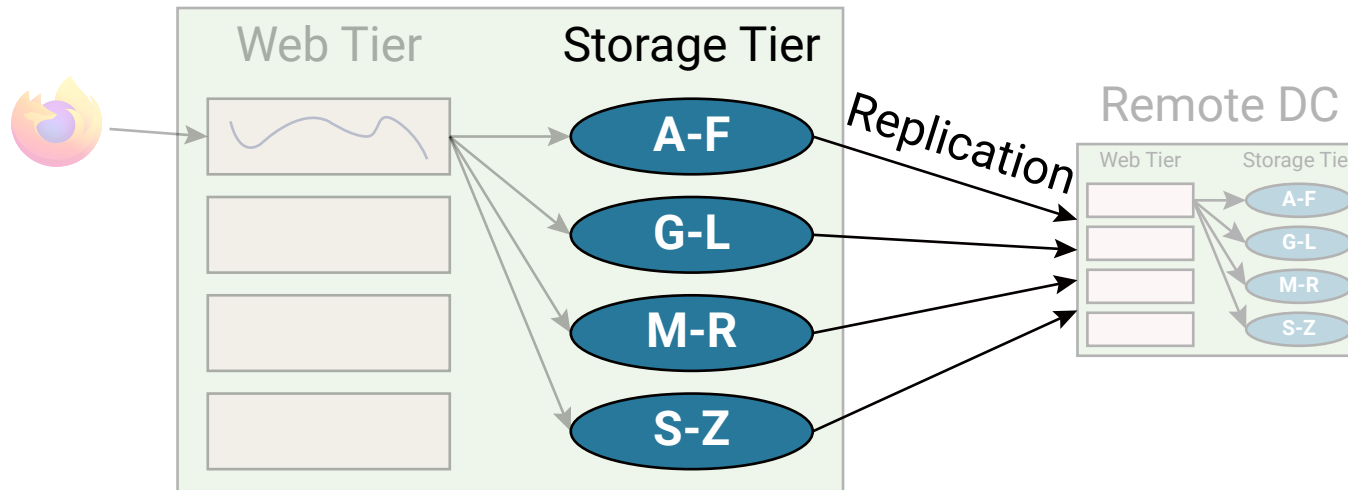
Inside the Datacenter



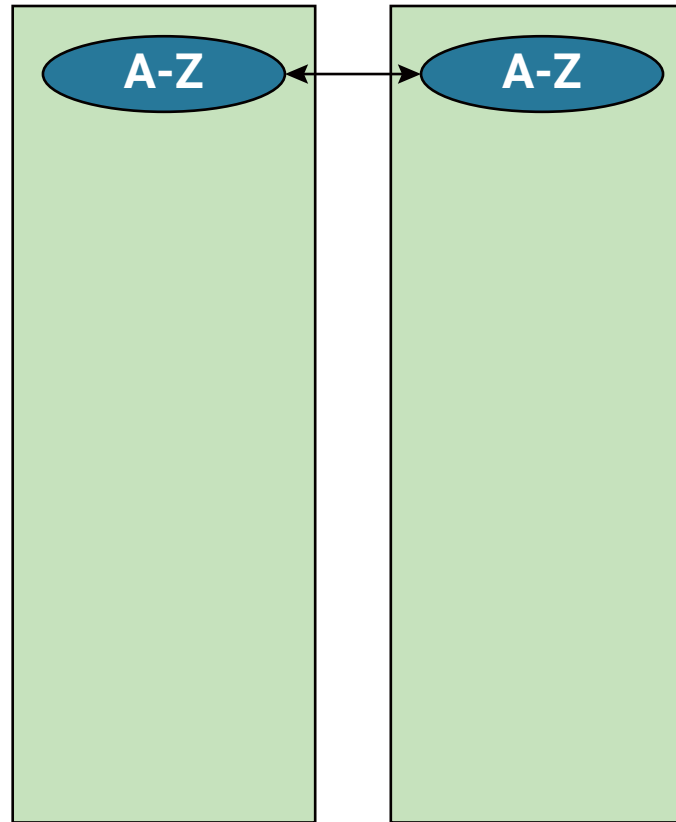
Inside the Datacenter



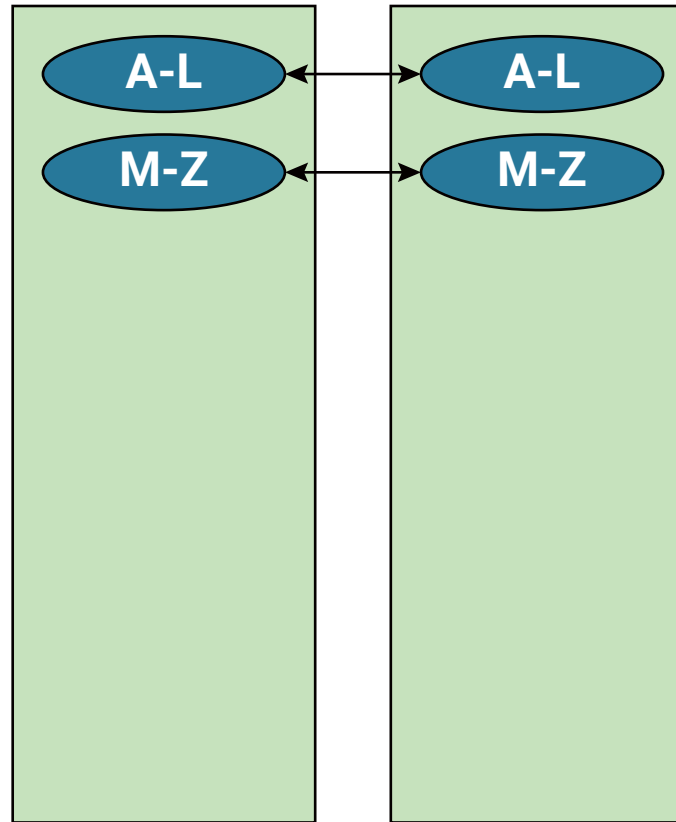
Inside the Datacenter



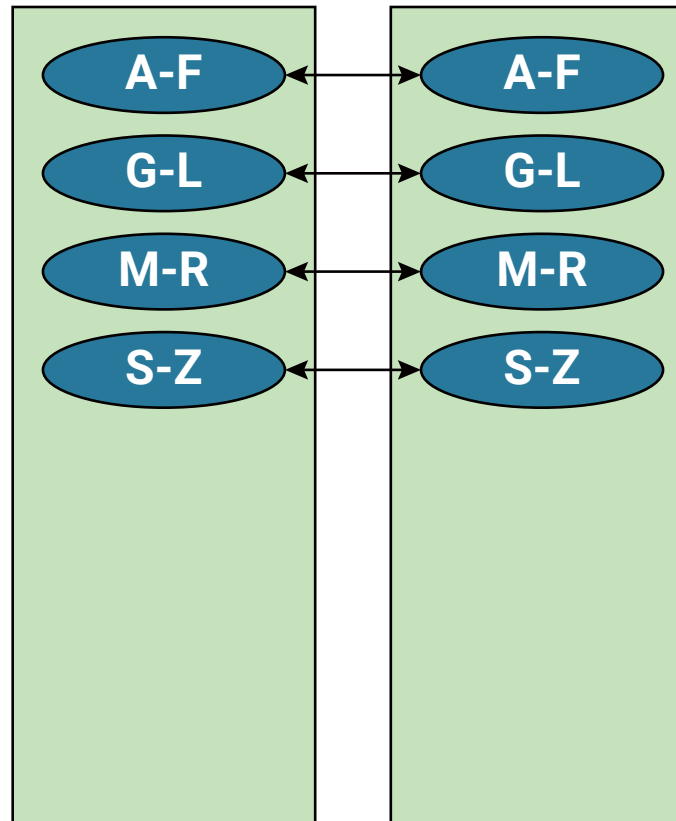
Scalability through Sharding



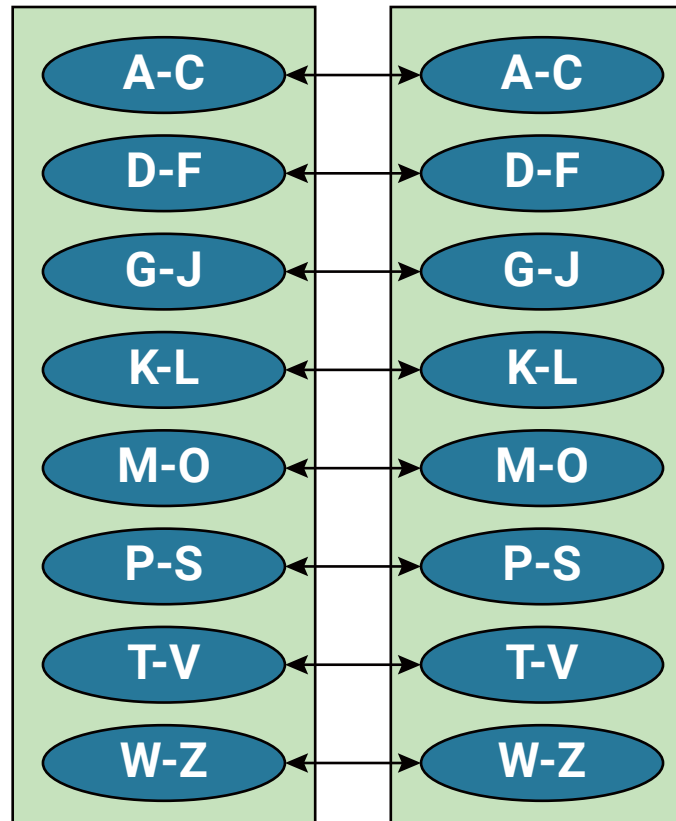
Scalability through Sharding



Scalability through Sharding



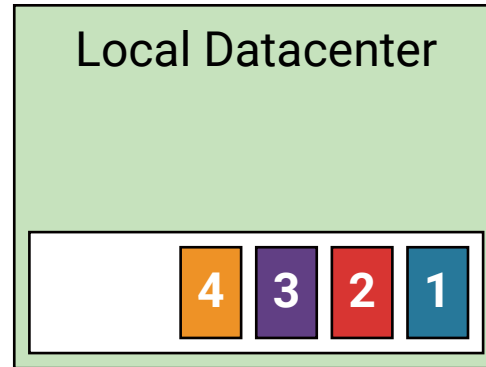
Scalability through Sharding



Bayou's Causal Consistency

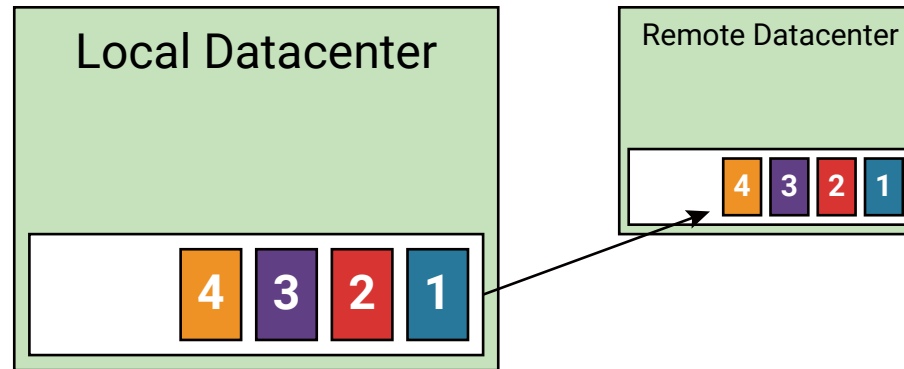


- Log-exchange based



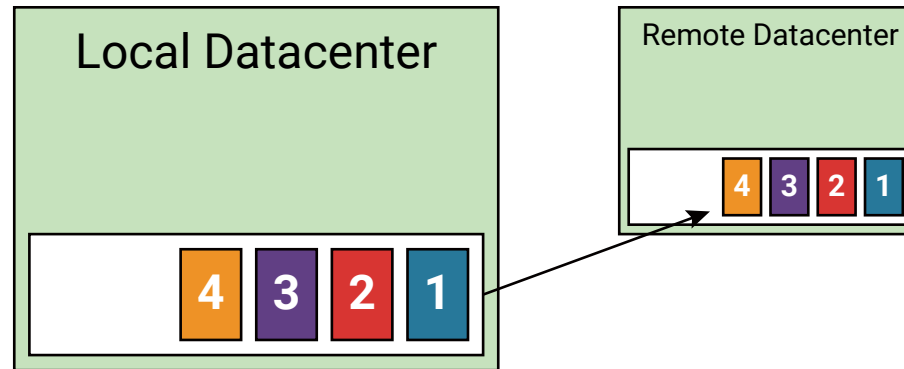
Bayou's Causal Consistency

- Log-exchange based



Bayou's Causal Consistency

- Log-exchange based



- Log is single serialization point within DC
 - ✓ Implicitly captures & enforces causal order

Sharded Log Exchange

- What happens if we use a separate log per shard?
- What happens if we use a single log?

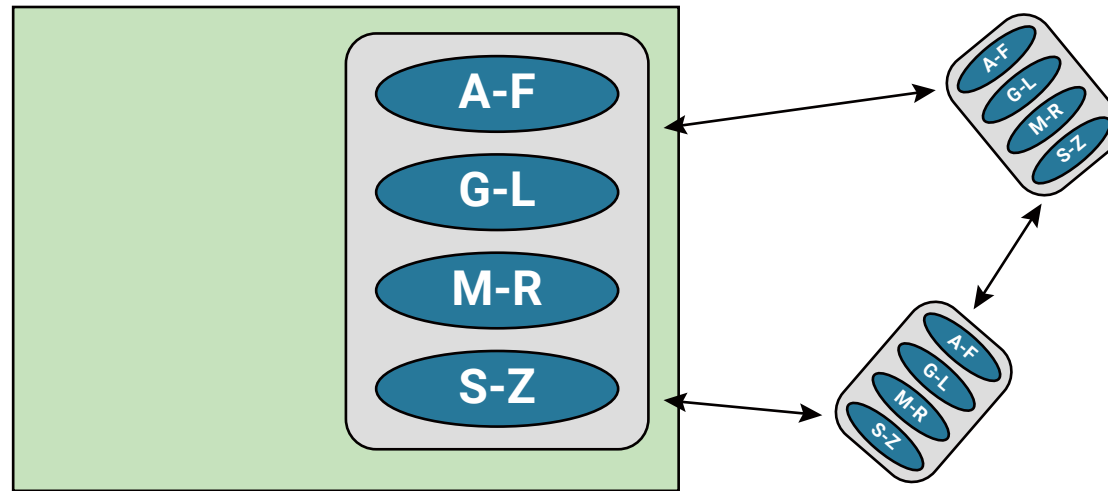
Scalability Idea

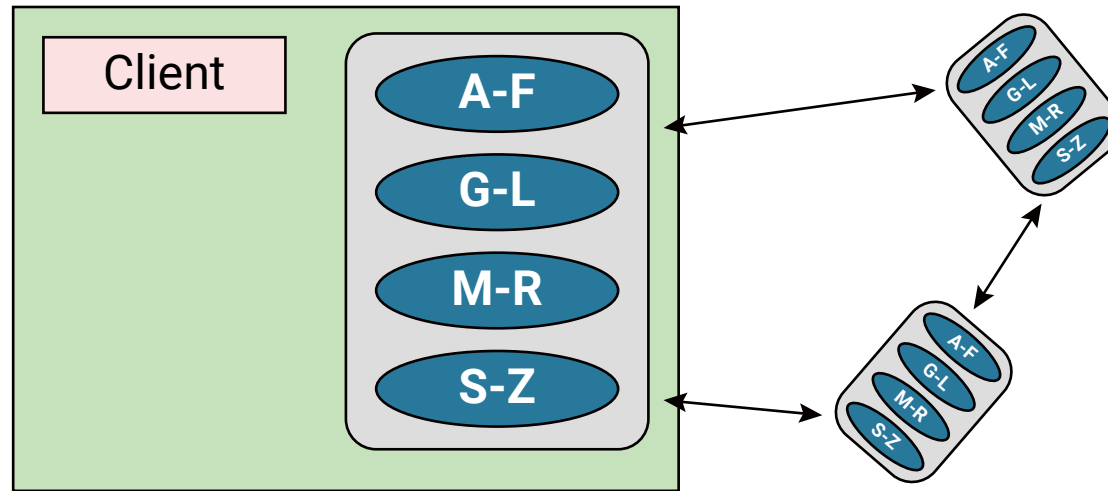


- Each key is associated with a monotonically increasing version number
- Capture causality with per-client dependency metadata

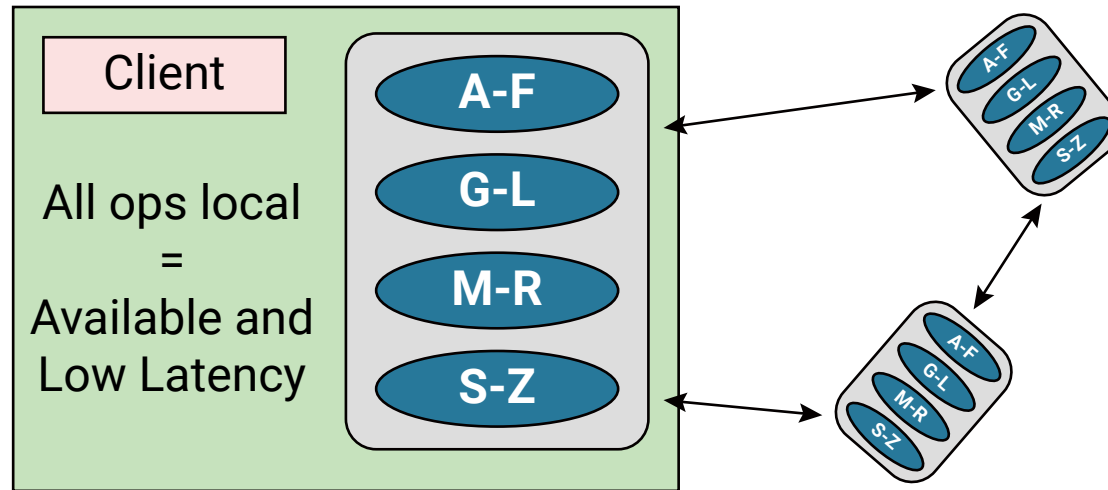
Client operations	Read key 1	Read key 2	Read key 3
Dependency metadata	Key 1 (version 100)	Key 2 (version 25)	Key 3 (version 50)

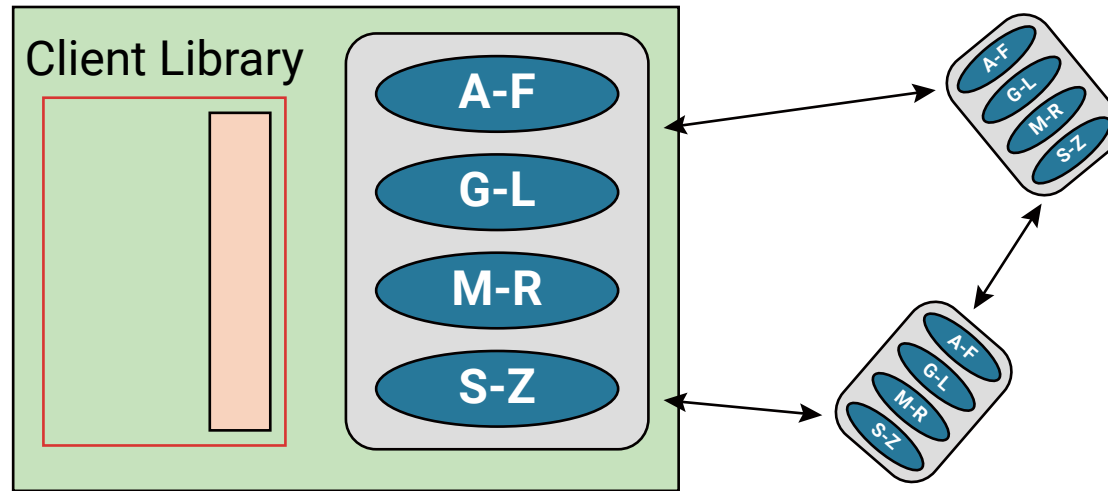
- Delay exposing replicated writes until all dependencies are satisfied in the datacenter

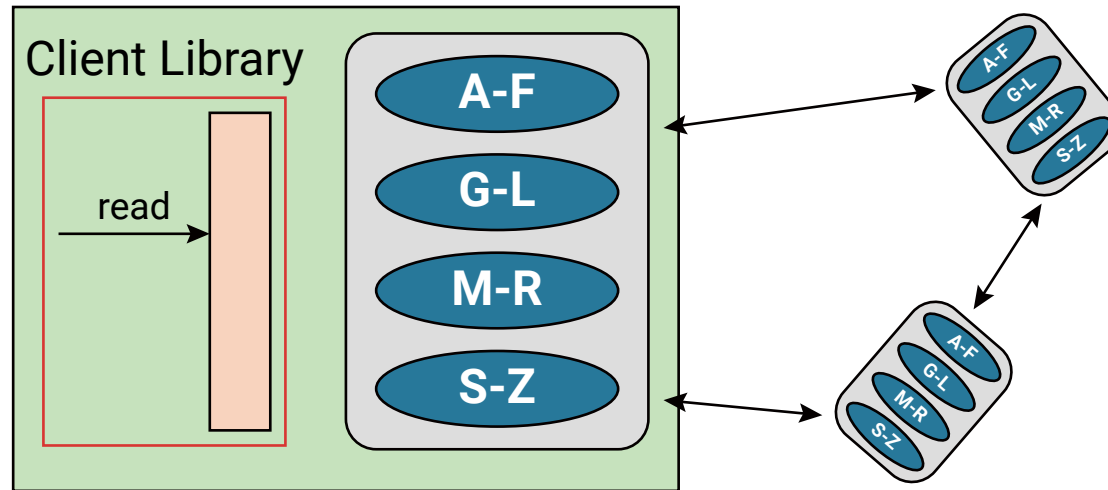


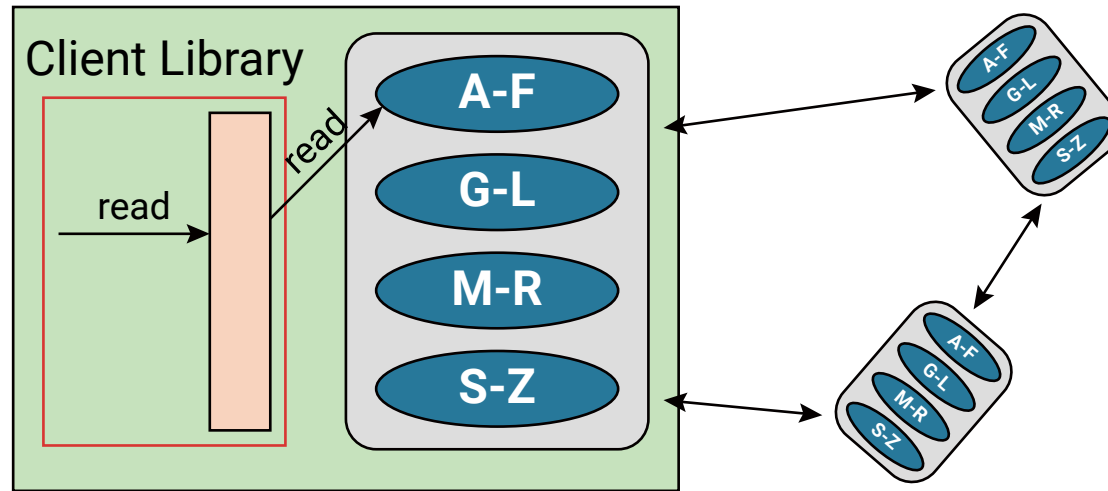


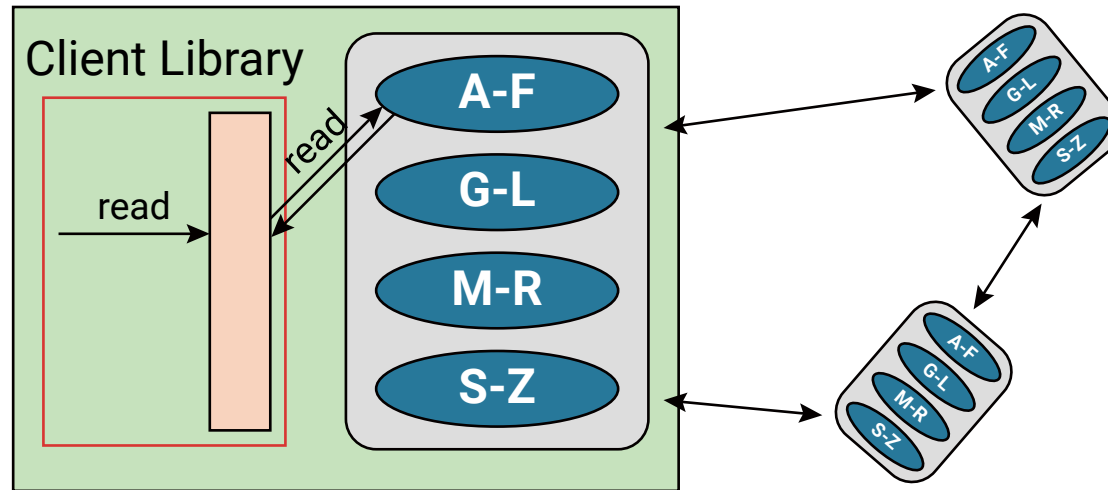
COPS Architecture

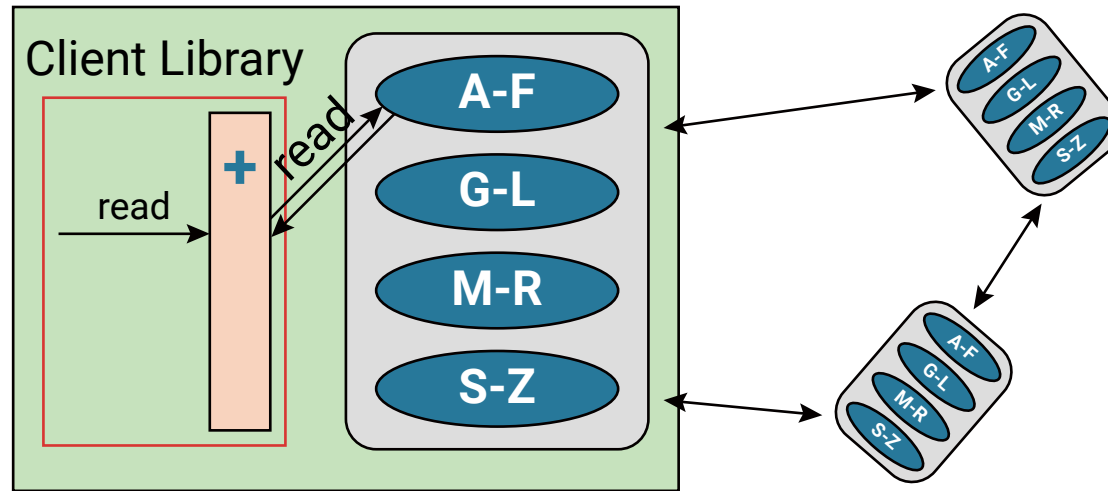


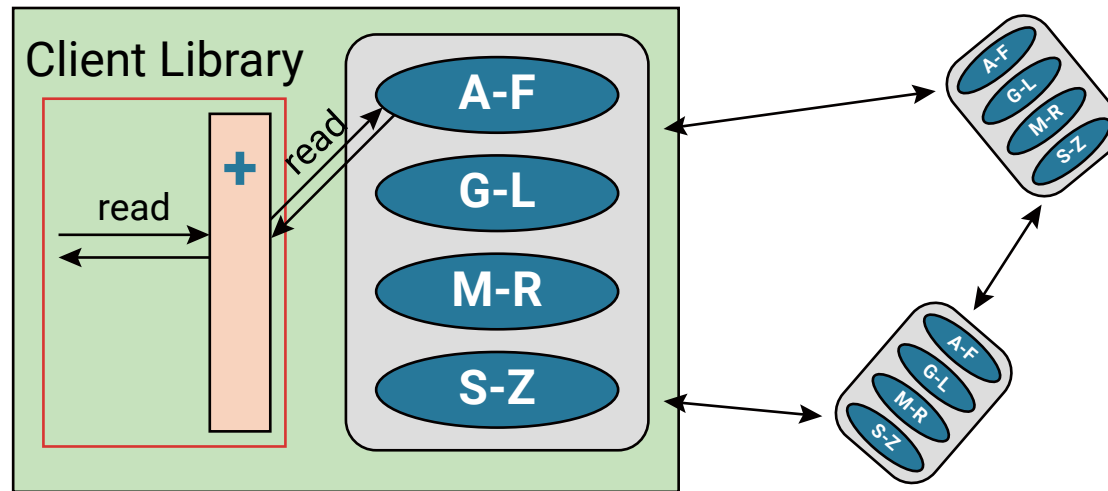


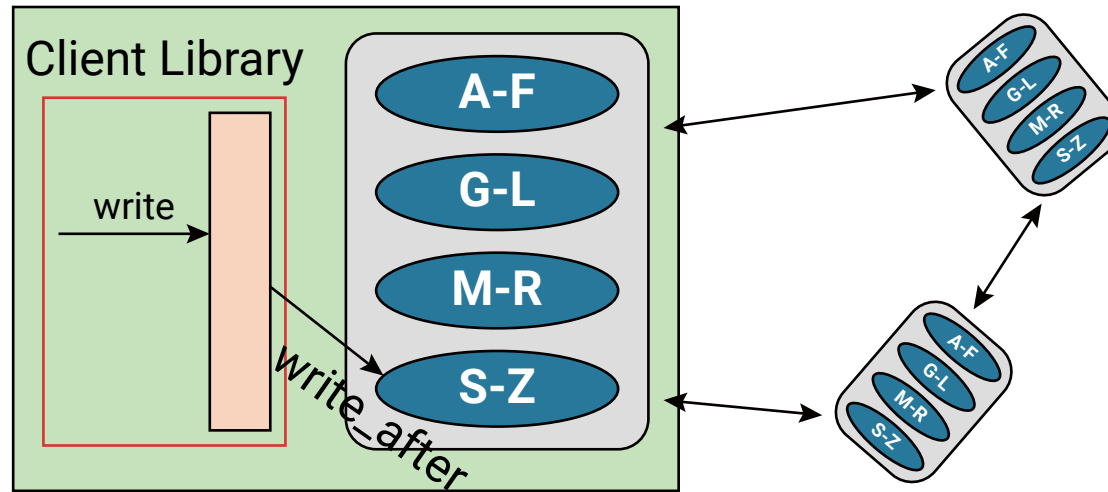




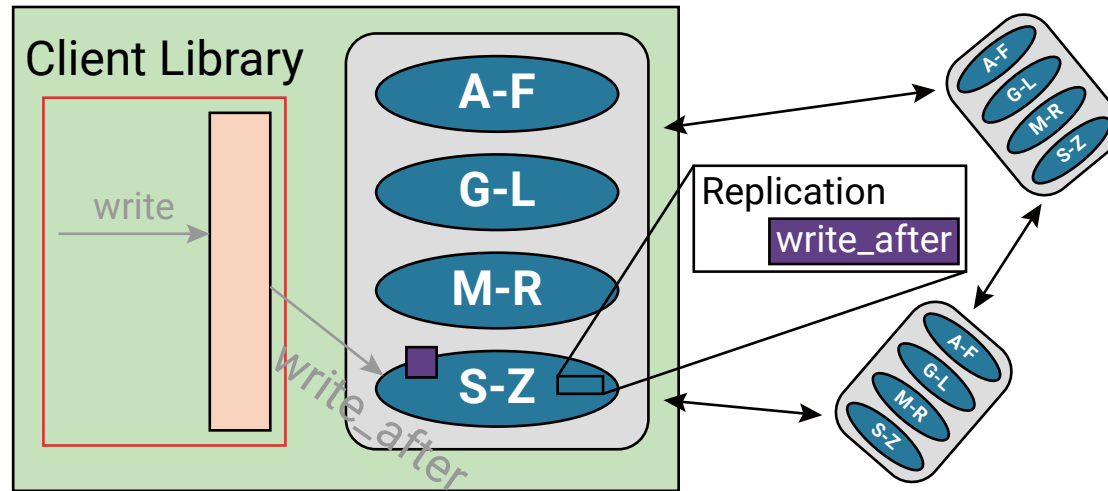




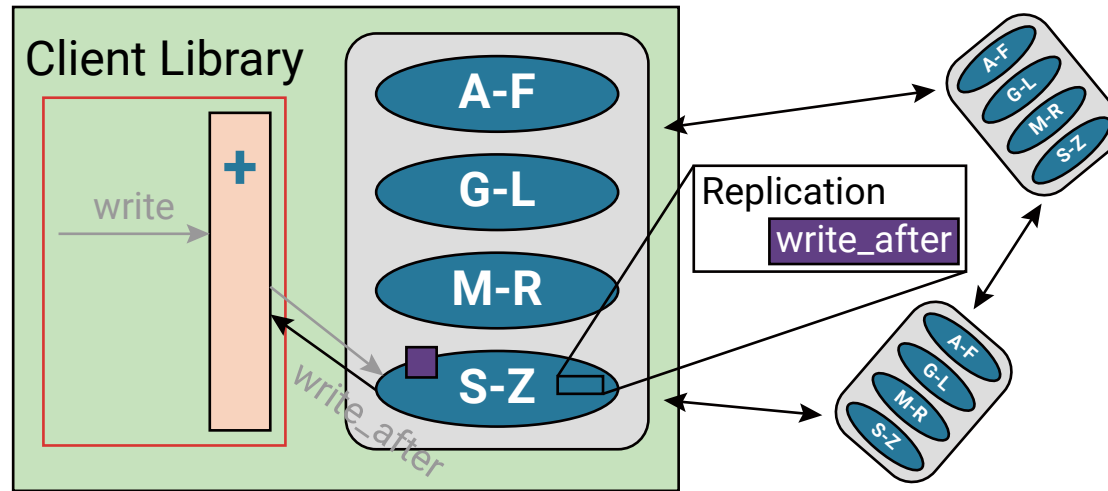




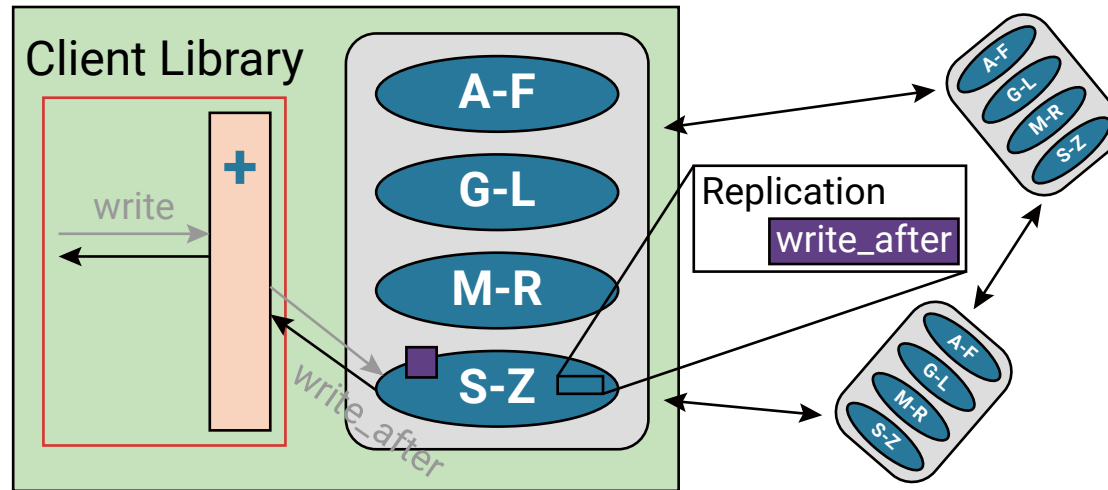
$\text{write_after} = \text{write} + \text{ordering metadata}$



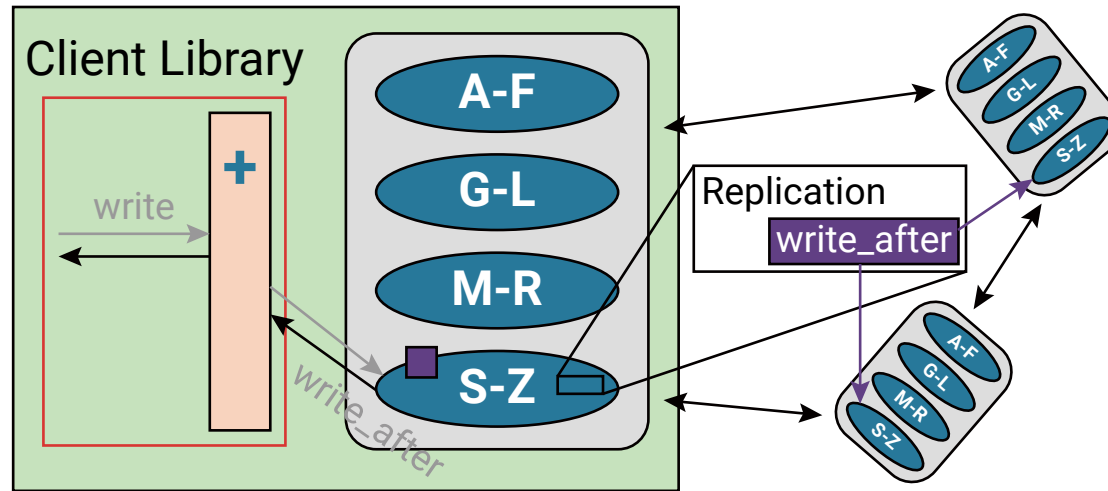
$\text{write_after} = \text{write} + \text{ordering metadata}$



$\text{write_after} = \text{write} + \text{ordering metadata}$

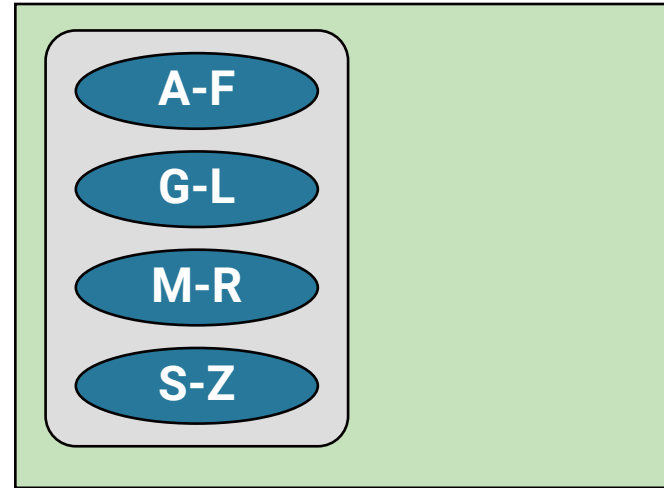


$\text{write_after} = \text{write} + \text{ordering metadata}$

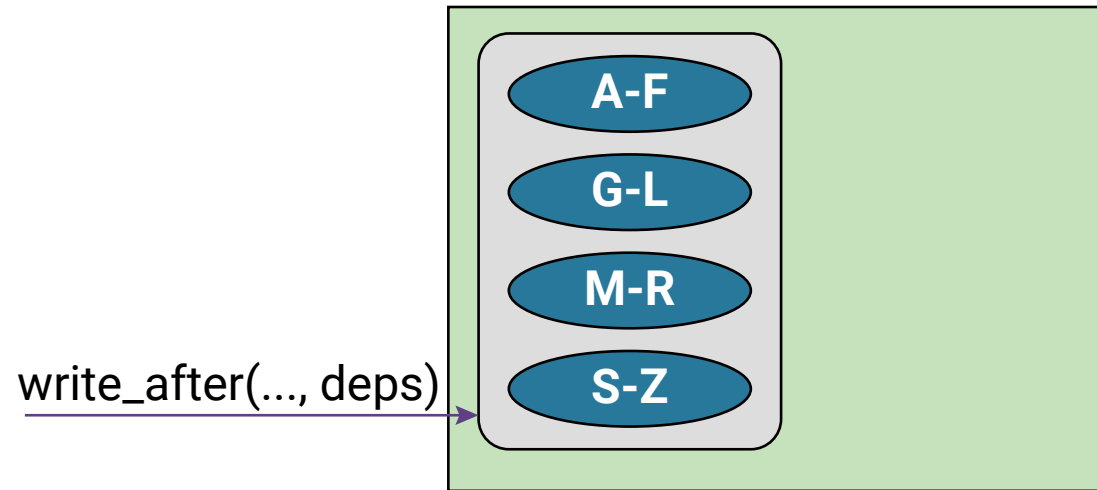


$\text{write_after} = \text{write} + \text{ordering metadata}$

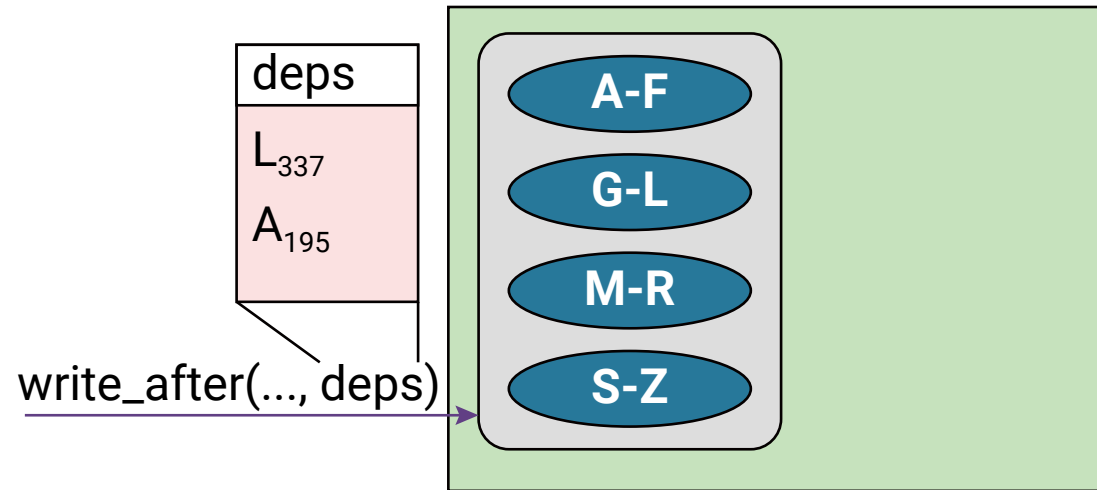
Replicated Write



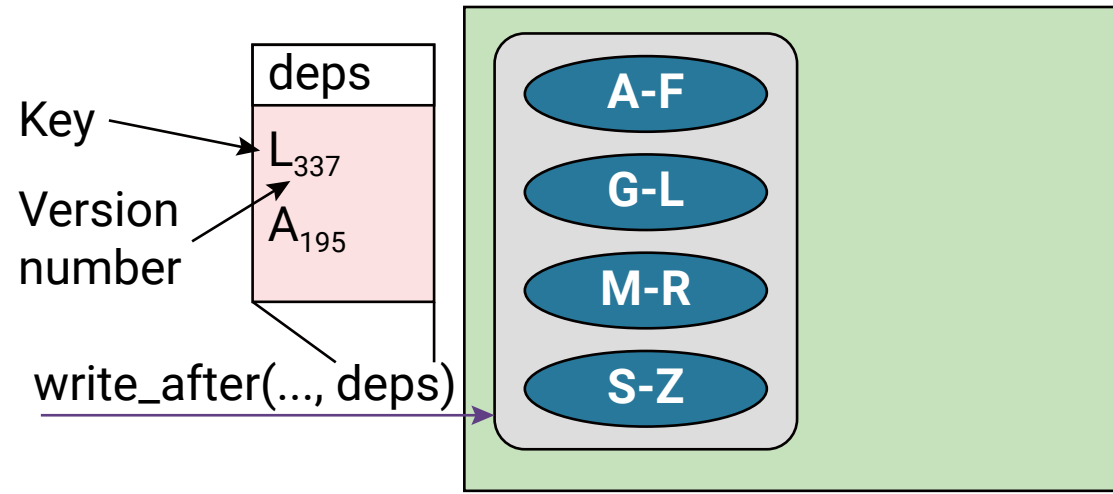
Replicated Write



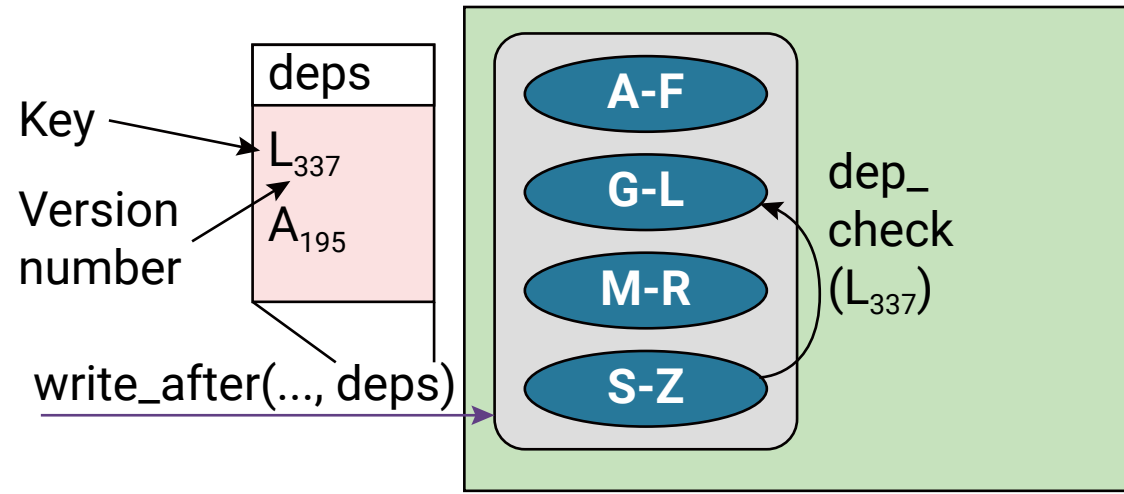
Replicated Write



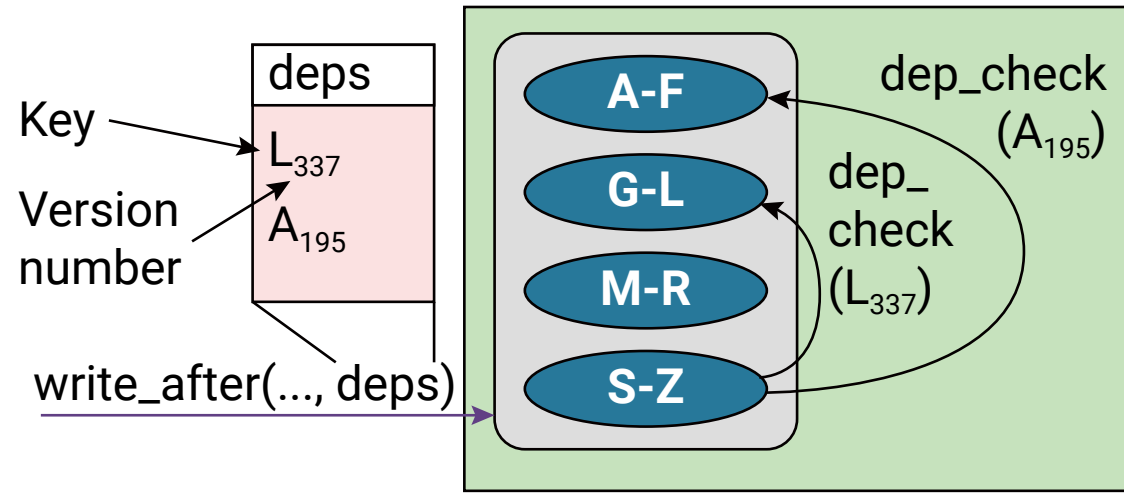
Replicated Write



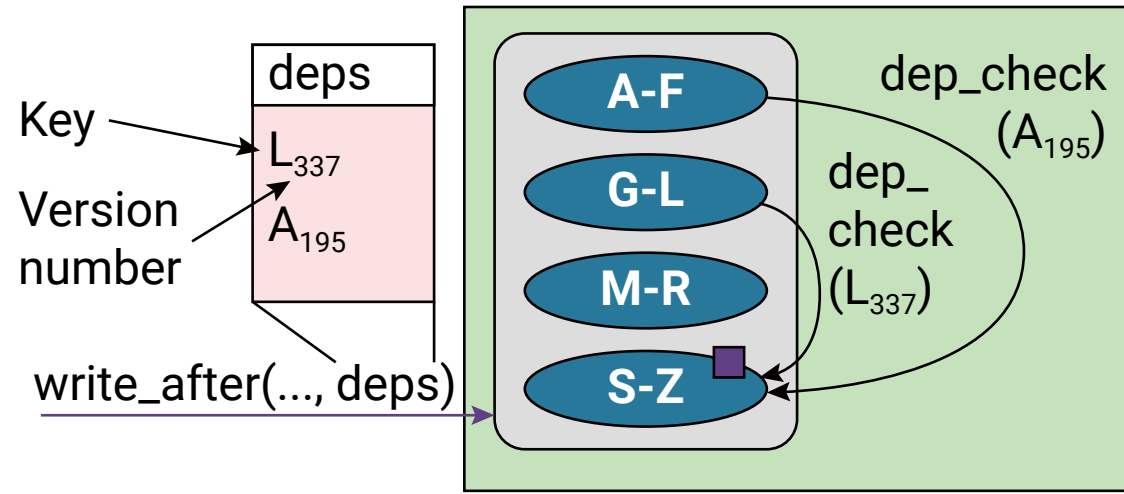
Replicated Write



Replicated Write



Replicated Write



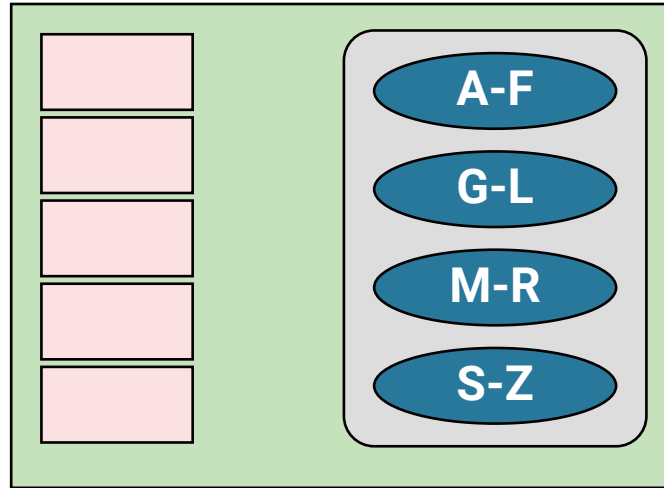
Exposing values after dep_checks
return ensures causal

Basic Architecture Summary

- All operations local, replicate in background
 - Availability and low latency
- Shard data across many nodes
 - Scalability
- Control replication with dependencies
 - Causal consistency

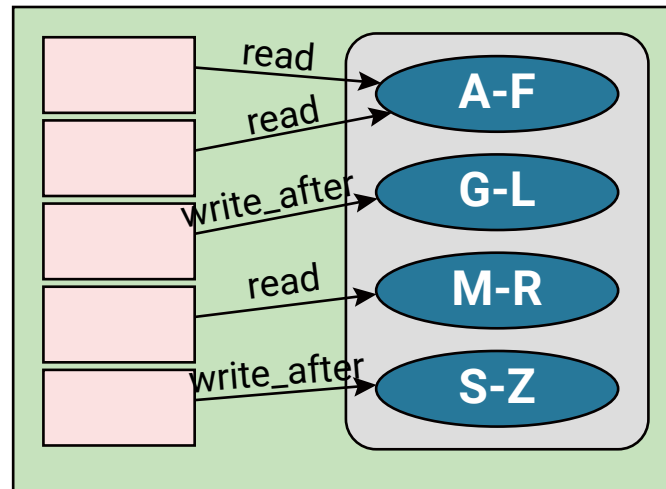


Scalable Causal+ From Fully Distributed Operation



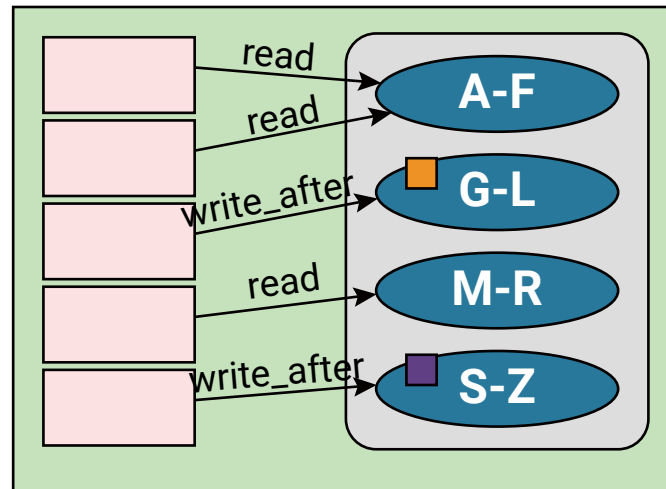


Scalable Causal+ From Fully Distributed Operation



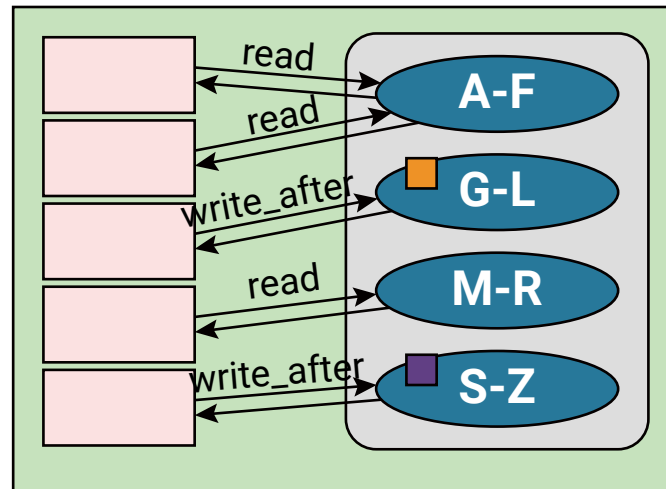


Scalable Causal+ From Fully Distributed Operation



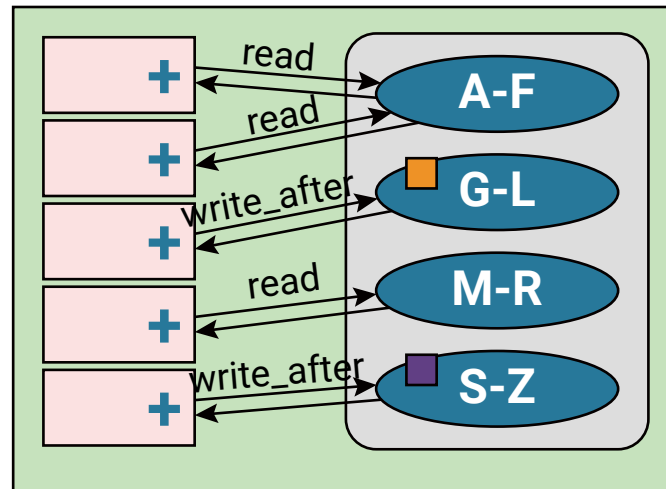


Scalable Causal+ From Fully Distributed Operation



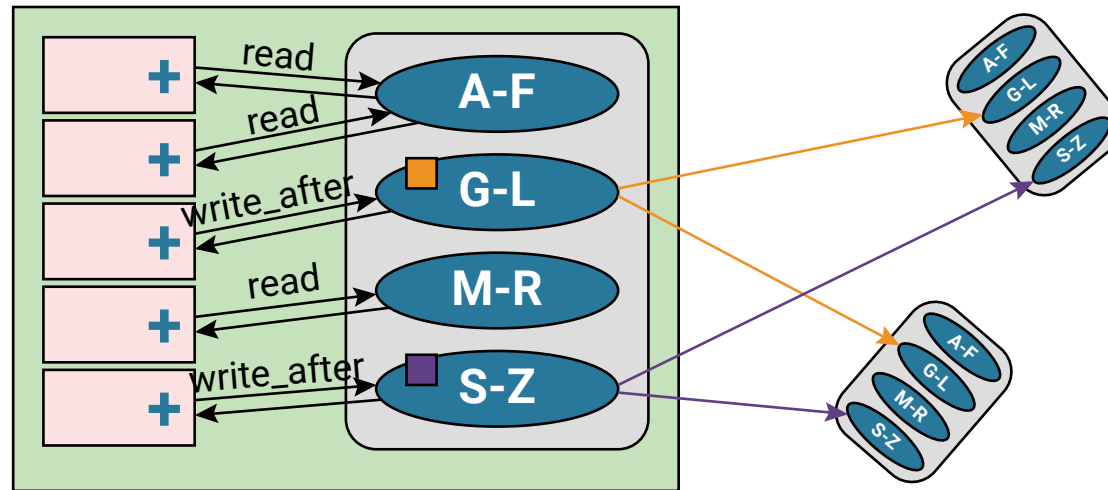


Scalable Causal+ From Fully Distributed Operation



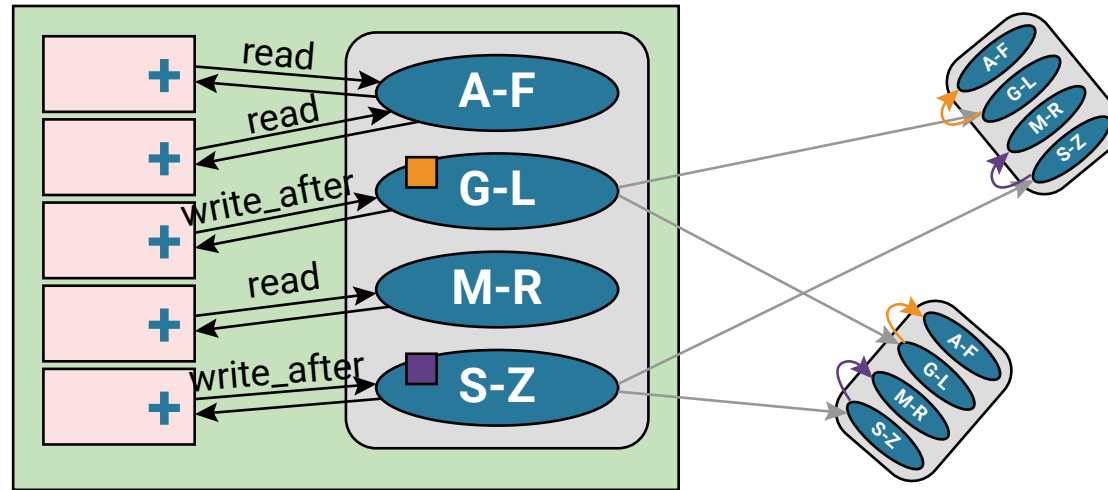


Scalable Causal+ From Fully Distributed Operation



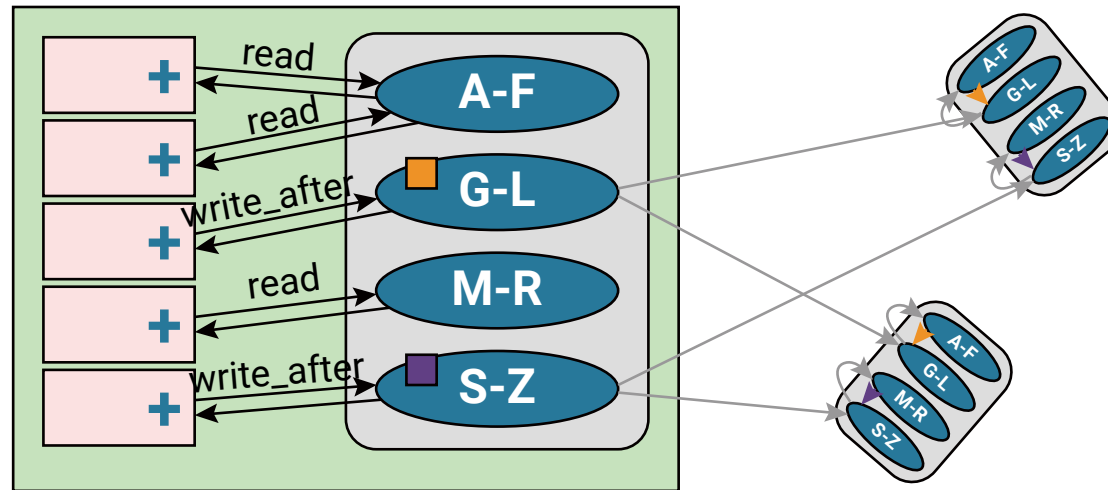


Scalable Causal+ From Fully Distributed Operation



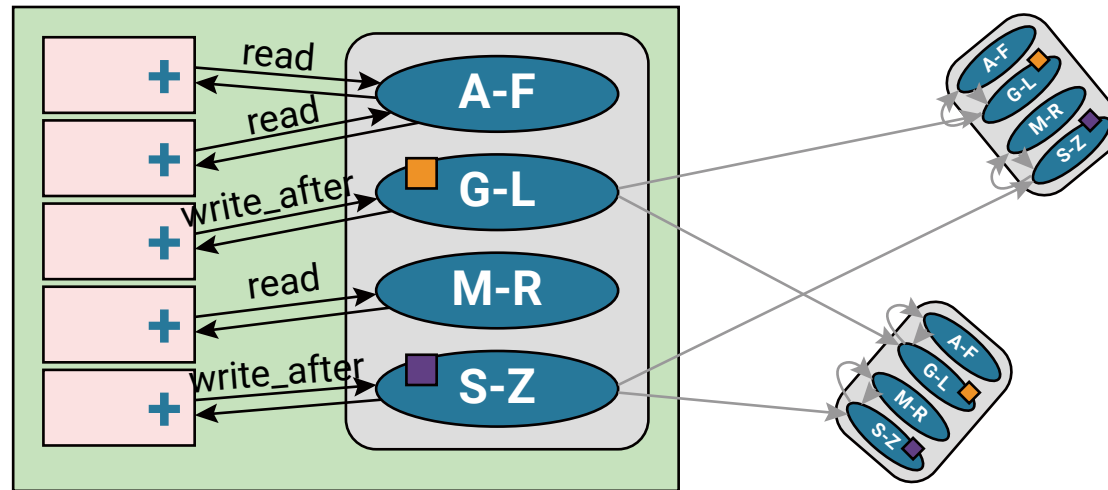


Scalable Causal+ From Fully Distributed Operation





Scalable Causal+ From Fully Distributed Operation



- Scalable causal consistency
 - Shard for scalable storage
 - Distributed protocols for coordinating writes and reads
 - Evaluation confirms scalability
- All operations handled in local datacenter
 - Availability
 - Low latency
- We're thinking scalably now!
 - Next time: scalable strong consistency

© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.